IT WORKSHOP | FULL-STACK WEB APPLICATION

# GROUP PROJECT DEMONSTRATION - STUDENT SUBMISSION

To be completed by the project team prior to evaluation

| | |
|---|---|
| **Group / Batch / Section:** DSAI 1st Semester | **Project:** NoteSphere - Smart Student Notes Sharing Platform |
| **Date / Time / Venue:** 26/10/2026 \| Time: _____<br>Venue: _____ | **Demonstration URL:** http://127.0.0.1:3000<br>Localhost; launcher may choose another free port. |
| **Repository / Commit:** Source maintained locally; repository / commit ID: _____ | **Hosting status:** Local demonstration. Cloud deployment has not been verified. |

## 1. GROUP MEMBERS & INDIVIDUAL CONTRIBUTIONS

| # | STUDENT NAME | ROLL NUMBER | PRIMARY ROLE / WORK DELIVERED |
|---|---|---|---|
| 1 | **Harmandeep Singh** | 261010234 | Frontend |
| 2 | **Aahaan Jain** | 261010201 | Frontend |
| 3 | **Kundan Pathak** | 261010242 | Database Management |
| 4 | **Sparsh Dwiwedi** | 261010290 | Backend |
| 5 | **Shreshth Sharma** | 261010286 | Testing and Deployment tools |

## 2. PROJECT SUMMARY

**Problem addressed & target users:** Academic notes are scattered across chats and personal folders. NoteSphere gives college students a searchable, organized library of PDFs published by administrators, with protected reading access and persistent personal collections.

**Account access:** Students register and sign in; administrators use a separate portal with invitation-based account creation. Profiles store academic preferences.

**Publication:** An admin uploads a validated PDF with subject, course, semester and tags. Published community notes become available to students and generate notifications; drafts and private notes stay restricted.

**Study workflow:** Signed-in students search/filter notes, preview PDFs using PDF.js, download, bookmark, organize collections, and write or update one review per note.

**Administration:** Admins manage uploads, student records, reports and academic catalog entries. Backend checks enforce roles and ownership.

## 3. TECHNOLOGY & IMPLEMENTATION CHECKLIST

**Front end:** Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4, custom responsive globals.css, Lucide icons and PDF.js. Accessible forms, dark/light themes and loading/error states.

**Back end:** Node.js runtime with Next.js API route handlers; TypeScript, Zod validation, file-type signature checks, protected PDF storage and student notifications.

**Database:** PostgreSQL via on-disk PGlite locally; SQL migrations, foreign keys, unique constraints and search indexes. Main entities: users, notes, files, reviews, bookmarks, collections and notifications.

**Authentication / authorization:** Custom salted scrypt passwords and database sessions with HttpOnly cookies. Server-side student/admin checks, origin validation and rate limiting; no Google OAuth or Supabase Auth.

**Testing / delivery:** Node test runner (4 unit tests), TypeScript checks and 17 isolated HTTP/database integration checks recorded as passed. Windows .cmd launcher; local offline use after dependency installation. Vercel is unverified.

## 4. DEMONSTRATION READINESS & STUDENT DECLARATION

- □ Application runs
- □ UI / API / DB flow shown
- □ CRUD operations shown
- □ Schema / ER diagram ready
- □ Input validation shown
- □ Team ready for viva
- □ Source code available
- □ Test data prepared

# FACULTY ASSESSMENT & LIVE DEMONSTRATION RECORD

For faculty evaluator use during project defense | Total: 100 marks

**Group / Project:** DSAI 1st Semester - NoteSphere
**Evaluator:** _____ **Evaluation date:** _____
**Scoring bands:** Excellent 90-100%; Good 75-89%; Satisfactory 60-74%; Limited 40-59%; Inadequate below 40%. Award whole marks from observed evidence.

## 5. EVALUATION CRITERIA & SCORING RUBRIC

| # | ASSESSMENT CRITERION | OBSERVABLE EVIDENCE / EXPECTATIONS | MAX | AWARDED |
|---|---|---|---|---|
| 1 | **Problem definition & scope** | Clear purpose, student/admin requirements, appropriate scope and coherent note-sharing workflow. | 10 | |
| 2 | **Front-end design & usability** | Layout, navigation, responsiveness, accessible forms and useful validation feedback. | 15 | |
| 3 | **Back-end implementation** | API handlers, business rules, validation, modular structure and safe errors. | 15 | |
| 4 | **Database design & operations** | Relational schema, constraints, indexed queries and persistent CRUD operations. | 15 | |
| 5 | **Full-stack integration** | Reliable browser-to-API-to-database flow and complete user journeys. | 15 | |
| 6 | **Security & reliability** | Authentication, server-side roles, protected files, safe inputs and secrets. | 10 | |
| 7 | **Testing & code quality** | Unit/integration cases, edge cases, readable source and documentation. | 10 | |
| 8 | **Live demo, teamwork & viva** | Balanced participation, clear individual explanations and accurate answers. | 10 | |
| | **TOTAL GROUP SCORE** | | **100** | ___ / 100 |

## 6. LIVE DEMONSTRATION & VIVA VERIFICATION LOG

**Create / retrieve / update / delete note-library data:** Outcome / remarks: _____

**Browser to API to database integration:** Outcome / remarks: _____

**Invalid input, denied access and error recovery:** Outcome / remarks: _____

**Source code, database schema and local startup:** Outcome / remarks: _____

## 7. INDIVIDUAL PARTICIPATION, VIVA NOTES & SIGN-OFF

**1. Harmandeep Singh (261010234) - Frontend**
Viva notes / adjustment / sign-off: _____

**2. Aahaan Jain (261010201) - Frontend**
Viva notes / adjustment / sign-off: _____

**3. Kundan Pathak (261010242) - Database Management**
Viva notes / adjustment / sign-off: _____

**4. Sparsh Dwiwedi (261010290) - Backend**
Viva notes / adjustment / sign-off: _____

**5. Shreshth Sharma (261010286) - Testing and Deployment tools**
Viva notes / adjustment / sign-off: _____

Evaluator signature: _____ Team representative: _____